

User Defined Data Structures: Stack in Python



by
Dr M Rajasekhara Babu

School of
Computer Science and Engineering
<https://www.rajababu.org>

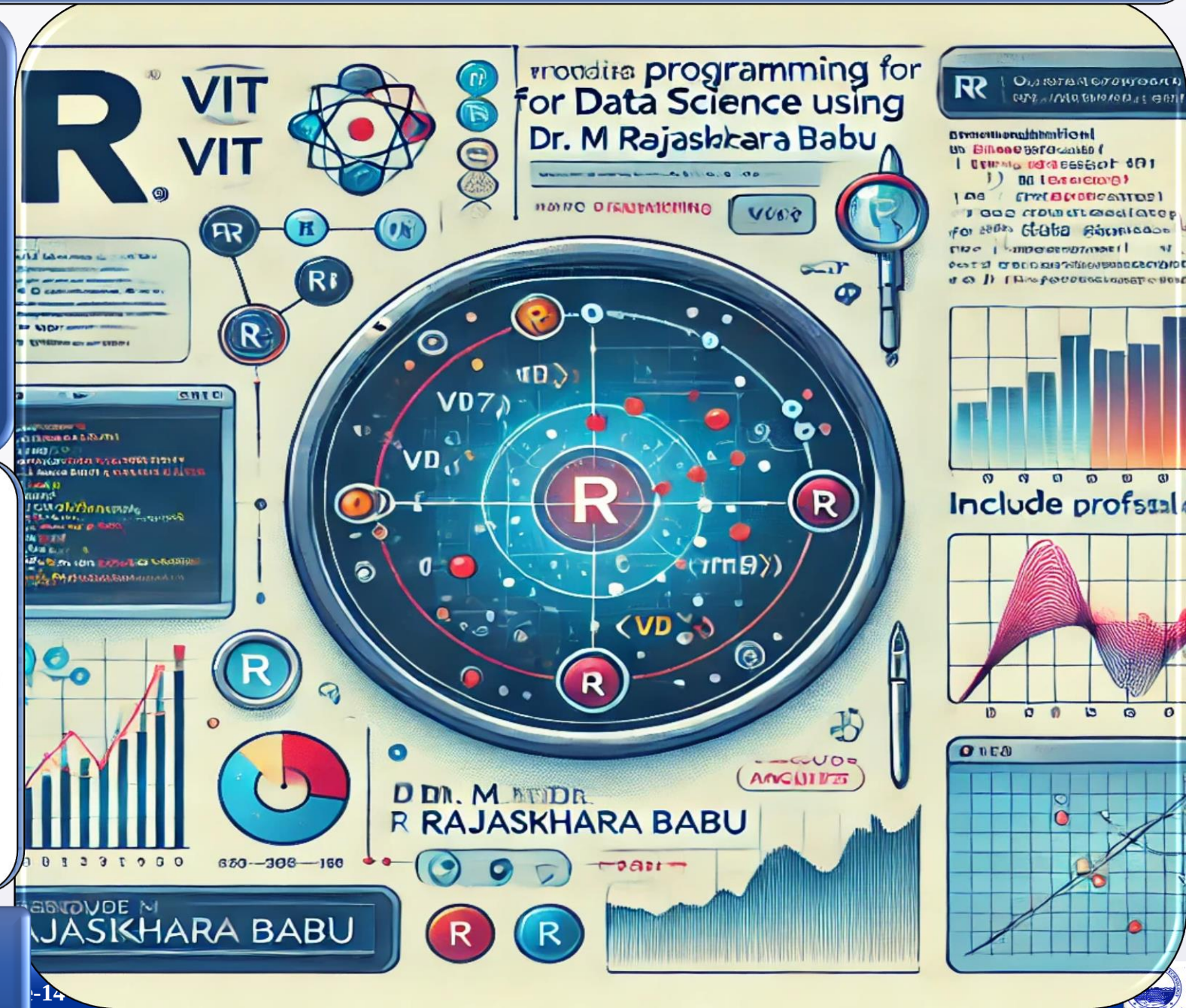


VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Vellore-632014, Tamil Nadu, India



Objectives & Teaching Learning Material



To explain the concept of stacks and the LIFO principle used in user-defined data structures



To demonstrate stack operations such as Push, Pop, Peek, isEmpty, and Display using Python



To implement and analyze a user-defined Stack class in Python and evaluate its practical applications and performance

Teaching + Learning

Material:

LCD, White board Marker, Presentation slides

Learning outcomes

Upon successful completion of this lecture, students will be able to:



Explain the structure and behavior of stacks and apply the LIFO principle in problem-solving scenarios



Implement stack operations using Python lists and develop a user-defined Stack class using object-oriented programming concepts



Analyze the efficiency and applicability of stack operations in real-world computing tasks such as function calls, undo mechanisms, and expression evaluation



Session Plan

Time in minutes	Topic	Learning Aid & Methodology	Faculty Approach	Student Activity	Skill and Competency Developed
05	Re-Cap	Quiz Presentation	Questions Organizes	Answers Identifies	Knowledge Understanding
10	Introduction to Data Structures and Stack Fundamentals	Presentation	Presentation	Listens	Remembering
10	Stack Operations and Applications	Explains	Explains	Notes	Understanding
15	Implementing Stacks in Python	Case Study	Organizes	Observes	Applying
10	Complexity Analysis and Real-World Uses	Discussion	Facilitates	Answers	Analyzing



After this session, students will be able to:

01

Understand the Stack

Explain the LIFO principle and stack structure.

02

Perform Operations

Execute Push, Pop, Peek, isEmpty, and Display operations.

03

Implement in Python

Build a stack using Python lists and a user-defined class.

04

Apply & Analyze

Solve real-world problems and analyze time complexity.

What is a Data Structure?

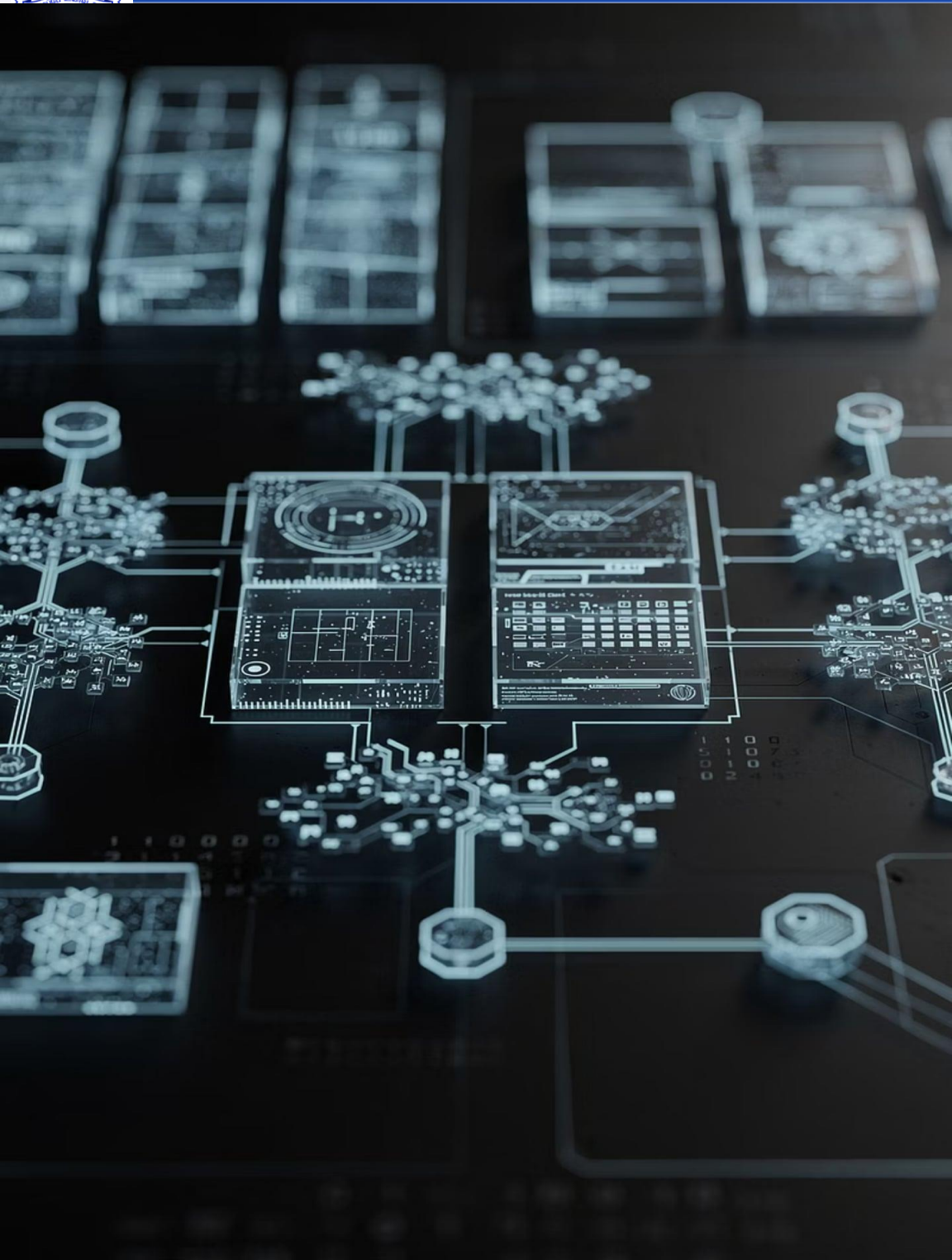
- A Data Structure is a way of organizing and storing data so that it can be accessed and modified efficiently.

Built-in Data Structures

- List
- Tuple
- Set
- Dictionary

User Defined Data Structures

- Stack
- Queue
- Linked List
- Tree
- Graph



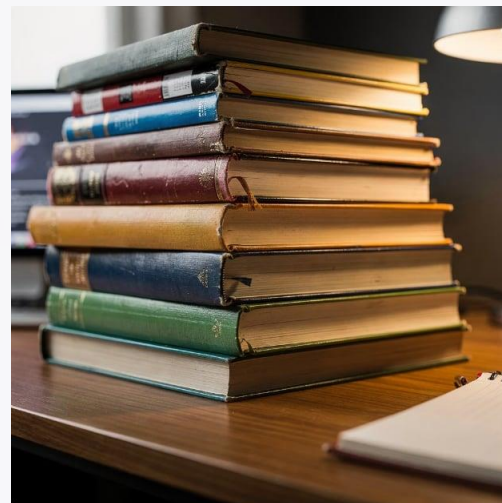
What is a Stack?

A Stack is a linear data structure that follows the **LIFO Principle — Last In, First Out**. The last element inserted is the first one removed.



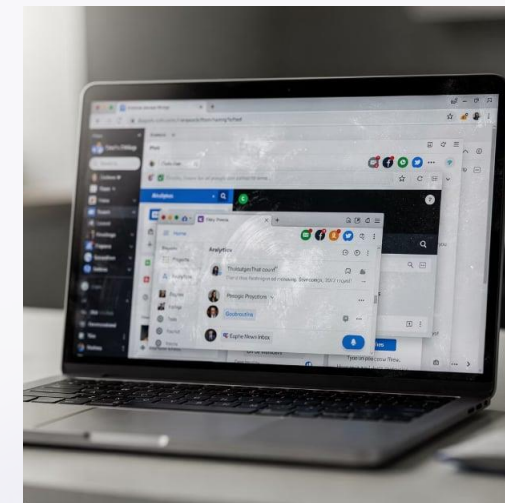
Stack of Plates

You always remove the top plate first — the last one placed is the first one taken.



Stack of Books

The book placed last sits on top and is the first to be picked up.



Browser History

The most recently visited page is the first returned when you press Back.

Real-Life Applications of Stack

All stack operations occur at one end called the TOP. Stacks appear everywhere in computing and daily life.



Undo Operation

MS Word, Photoshop, and VS Code use stacks to track and reverse recent actions.



Expression Evaluation

Compilers and calculators use stacks for evaluating expressions and matching parentheses.



Call Stack

Programs use a call stack to manage function calls and returns during execution.

Stack Operations

A stack supports six fundamental operations. Most execute in constant time **$O(1)$** .

Push

Insert a new element at the TOP.

Pop

Remove the top element from the stack.

Peek

View the top element without removing it.

isEmpty

Check whether the stack is empty.

Size

Find the number of elements in the stack.

Display

Show all stack contents.

PUSH Operation

Inserts a new element at the TOP of the stack.

Before Push: TOP $\rightarrow$ 30, 20, 10

Push(40)

After Push: TOP $\rightarrow$ 40, 30, 20, 10

POP Operation

Removes the element from the TOP of the stack.

Before Pop: TOP $\rightarrow$ 40, 30, 20, 10

Pop()

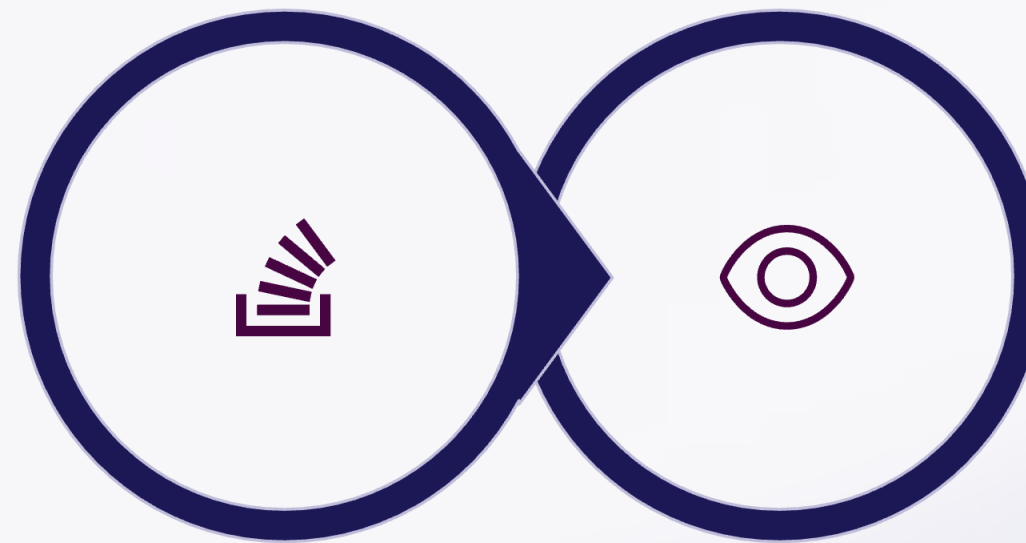
After Pop: TOP $\rightarrow$ 30, 20, 10

Deleted = 40

- Both Push and Pop operate exclusively at the TOP — this is the defining characteristic of a stack.

PEEK Operation

Peek returns the top element without removing it. It is a non-destructive read of the stack's top.



Stack:
[10,20,30,40]

Peek → 40

Unlike Pop, Peek leaves the stack intact — useful when you need to inspect the top value before deciding whether to remove it.

Stack Using Python Lists

Python lists can act as stacks directly. Use `append()` to push and `pop()` to remove the top element.

Push Elements

```
stack = []
stack.append(10)
stack.append(20)
stack.append(30)
print(stack)
# Output: [10, 20, 30]
```

```
stack = []
if len(stack) == 0:
    print("Stack is Empty") # Output: Stack is Empty
```

Pop & Peek

```
stack = [10, 20, 30]
item = stack.pop()
print(item)    # Output: 30
print(stack)   # Output: [10, 20]

# Peek
print(stack[-1]) # Output: 20
```

User Defined Stack Class

Python classes let us create a fully encapsulated stack data structure with clear advantages over raw lists.



Encapsulation

Data and methods bundled together.



Reusability

Create multiple stack objects from one class.



Better Organization

Clean, object-oriented implementation.

```

1  class Stack:
2      def __init__(self):
3          self.top = -1
4          self.data = []
5
6      def push(self, x):
7          if self.top == -1:
8              self.data.append(x)
9              self.top = 0
10         else:
11             self.data.append(x)
12             self.top += 1
13
14     def pop(self):
15         if self.top == -1:
16             return -1
17         else:
18             self.data.pop()
19             self.top -= 1
20
21     def peek(self):
22         if self.top == -1:
23             return -1
24         else:
25             return self.data[self.top]
26
27     def is_empty(self):
28         return self.top == -1
29
30     def __str__(self):
31         return str(self.data)
32
33     def __len__(self):
34         return len(self.data)
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Building the Stack Class: Init & Push

We start by defining the class with an `__init__` method that creates an empty list, then add a `push()` method.

`__init__` Method

```
class Stack:  
    def __init__(self):  
        self.stack = []
```

Creates an empty stack whenever an object is instantiated.

`push()` Method

```
def push(self, item):  
    self.stack.append(item)
```

Adds an element to the top of the stack using `append()`.

Building the Stack Class: Pop & Peek

Both methods include an **empty stack check** before performing the operation to prevent errors.

pop() Method

```
def pop(self):  
    if len(self.stack) == 0:  
        return "Stack Empty"  
    return self.stack.pop()
```

Removes and returns the top-most element.

peek() Method

```
def peek(self):  
    if len(self.stack) == 0:  
        return "Stack Empty"  
    return self.stack[-1]
```

Returns the top element without deleting it.

The full implementation with a `display()` method and a working demonstration:

```
class Stack:

    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)

    def pop(self):
        if len(self.stack) == 0:
            return "Stack Empty"
        return self.stack.pop()

    def peek(self):
        if len(self.stack) == 0:
            return "Stack Empty"
        return self.stack[-1]

    def display(self):
        print(self.stack)

s = Stack()

s.push(10)
s.push(20)
s.push(30)

s.display()

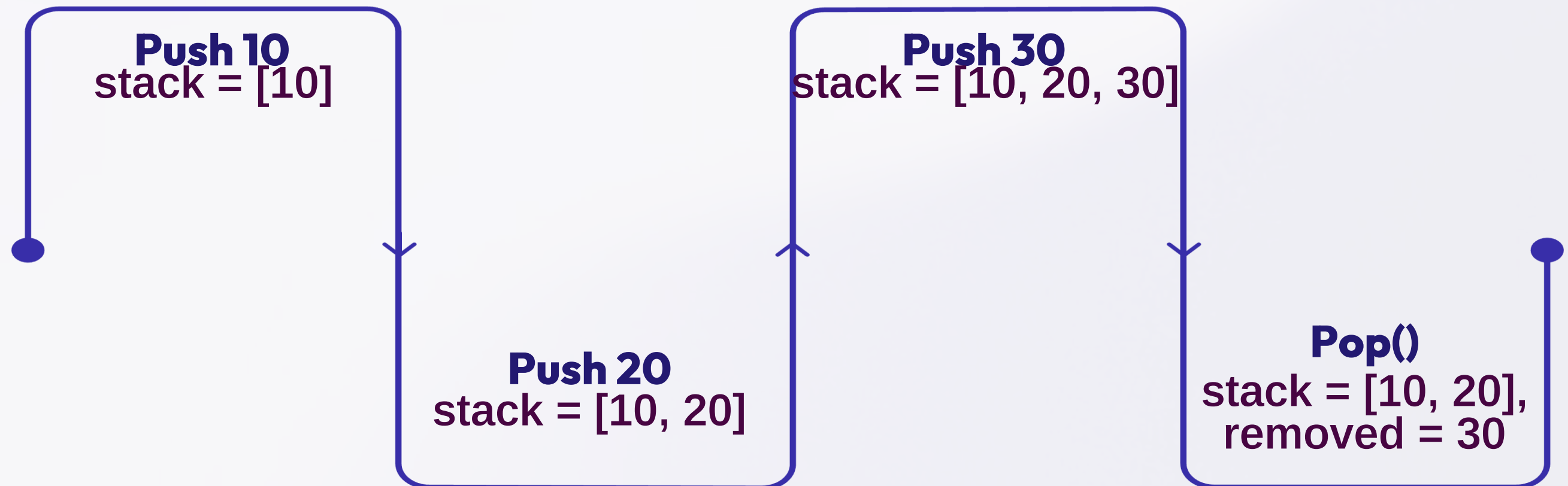
print("Top =", s.peek())

print("Deleted =", s.pop())

s.display()
```

Running the complete Stack class program produces the following output, demonstrating the **LIFO principle** in action:

```
[10, 20, 30]  
Top Element = 30  
Deleted = 30  
[10, 20]
```





Time Complexity of Stack Operations

One of the greatest strengths of the stack is its constant-time performance for most operations. Only Display is linear.

$O(1)$

Push

Constant time insertion at the top.

$O(1)$

Pop

Constant time removal from the top.

$O(1)$

Peek

Constant time top element access.

$O(n)$

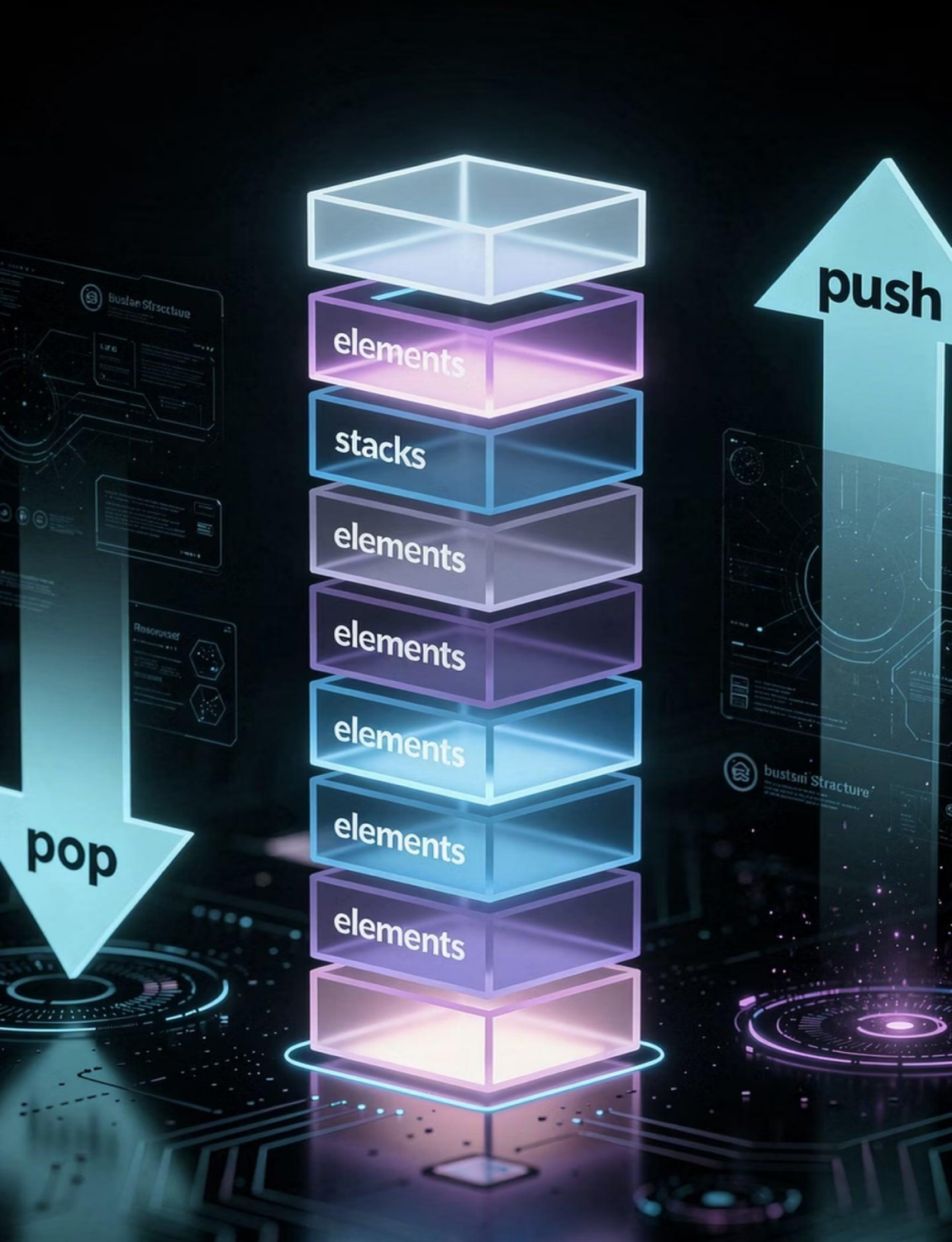
Display

Linear time to traverse all elements.

- Most stack operations execute in constant time — making stacks highly efficient for insertion and deletion.



Applications of Stack in Computing



Function Call Stack

Programs manage `main()` → `function1()` → `function2()` using a call stack that unwinds on return.

Undo Operation

MS Word, Photoshop, and VS Code push every action; Ctrl+Z pops the last one.

Browser Back Button

Each visited page is pushed; pressing Back pops to the previous URL.

Expression Evaluation & Parentheses Matching

Compilers use stacks to evaluate $(A+B)*C$ and verify balanced symbols `()` `{}` `[]`.

Advantages & Limitations of Stack

✓ Advantages

- Easy to implement
- Fast insertion and deletion — $O(1)$
- Less memory overhead
- Useful for recursion
- Efficient in backtracking problems

✗ Limitations

- Only the top element is accessible
- Restricted access to middle elements
- Cannot search efficiently
- Overflow possible for very large data

Practice Exercises

Apply what you've learned with these hands-on exercises:

1

Basic Stack Operations

Create a stack, push 10, 20, 30, and display all elements.

2

Pop & Display

Perform one Pop operation on the stack and display the remaining elements.

3

Implement Peek

Write a `peek()` method and test it on a stack with multiple elements.

4

Menu-Driven Program

Create a menu-driven stack program with options: 1. Push 2. Pop 3. Peek 4. Display 5. Exit

Topics Covered — Summary

Data Structures
Built-in vs. User
Defined

Complexity
 $O(1)$ for most
operations



Stack & LIFO
Concept and
representation

Operations
Push, Pop, Peek,
isEmpty, Display

**Python
Implementation**
Lists and user-defined
class



After completing this topic, students will be able to:

→ **Explain Stack & LIFO**

Describe the structure and behavior of a stack.

→ **Implement Using Python Lists**

Use `append()` and `pop()` to build a functional stack.

→ **Design a Stack Class**

Build an object-oriented, reusable Stack class with all methods.

→ **Apply & Analyze**

Solve real-world problems and evaluate $O(1)$ stack performance.

- **Linear Data Structures**
 - Sequential organization of data
 - Introduction to Queue and Priority Queue concepts
- **Queue Fundamentals**
 - Definition of Queue
 - FIFO (First In, First Out) principle
 - Real-life examples of queue systems
- **Queue Operations**
 - Enqueue
 - Dequeue
 - Front
 - Rear
 - isEmpty
 - Display
- **Queue Implementation Using Python**
 - Queue representation
 - Queue implementation using Python lists
 - User-defined Queue class
- **Applications of Queue**
 - Ticket reservation systems
 - Printer queues
 - CPU scheduling
 - Network transmission systems
- **Priority Queue Fundamentals**
 - Definition and characteristics
 - Priority-based processing
 - Difference between Queue and Priority Queue
- **Priority Queue Implementation in Python**
 - Tuple-based storage
 - Sorting by priority
 - User-defined Priority Queue class
- **Applications of Priority Queue**
 - Operating systems
 - Networking
 - Healthcare systems
 - Artificial Intelligence
 - Cloud computing applications
- **Queue vs Priority Queue**
 - Processing mechanisms
 - Operational comparison
 - Appropriate use cases
- **Time Complexity and Advantages**
 - Complexity analysis
- Advantages and limitations of Queue and Priority Queue

Reference

Alvaro Fuentes, Become a Python Data Analyst – By Packt Publishing (2018)

Bharti Motwani, Data Analytics using Python – By Wiley (2020)

Jules S. Damji, Learning Spark: Lightning-Fast Data Analytics, Second Edition – By Shroff/O'Reilly (2020)

Data Science and Machine Learning using Python –
10 August 2022, MGH, 2022